

Multi-Processing Camera Streaming & Real-Time AI Framework

1. Abstract

The Slugbotics topside vision infrastructure manages three high-definition video streams and handles real-time computer vision inference on the primary stream via an asynchronous, multi-processing architecture. High-resolution 1080p frame buffers are transferred across isolated process boundaries using atomic, zero-copy shared memory blocks to eliminate resource contention caused by Python's Global Interpreter Lock (GIL). A dedicated software watchdog monitor isolates individual stream failures and executes autonomous camera recovery without impacting overall pilot control loops.

2. Design Rationale & Software Control Architecture

2.1 Trade Off's & Limitations:

While C++ extensions for image decoding (`cv2.read()`) and inference (`model.predict()`) release the GIL during execution, the surrounding Python framework required for frame handling and orchestration forces severe lock contention under multi-threaded models. Moving operations to isolated operating system processes achieves true multi-core parallel processing without thread blockage.

2.1.1 Shared Memory Standard Python inter-process pipes rely on object serialization (`pickle`), creating severe CPU overhead when passing uncompressed 1080p images ($1920 * 1080 * 3 \sim 6.2$ MB). By memory-mapping NumPy arrays directly onto pre-allocated raw byte blocks in RAM and using in-place pointer updates (`np.copyto()`), deep array allocations are entirely eliminated.

2.1.2 Race Conditions & Fixed Allocations: If synchronization flags (`frame_tick_X`) are mismanaged between processes, workers can read partially overwritten data or re-read stale frames. Additionally, buffers are pre-allocated at a rigid byte size based on expected image geometry, preventing dynamic resolution scaling on the fly without unlinking memory blocks and freezing the stream. Offloading our ~ 6.4 GFLOPS model to external coprocessors (Edge TPUs/GPUs) was rejected to eliminate physical bus bottlenecks (PCIe/USB data transfer latency). Performing inference on the host CPU takes 15–17 ms, keeping frames in host memory to maximize cache-locality for subsequent layout tiling (`combine()`) and pilot HUD overlays.

3. Autonomous Watchdog Monitor:

To mitigate stream hangs where a camera driver stalls but remains registered in the OS, an autonomous loop monitors the frame update interval (Δt). If a stream stops providing data for > 2.0 seconds ("Soft Stall"), the worker process breaks execution, runs a forced `video.release()`, and re-initializes the handle without disrupting pilot control loops.

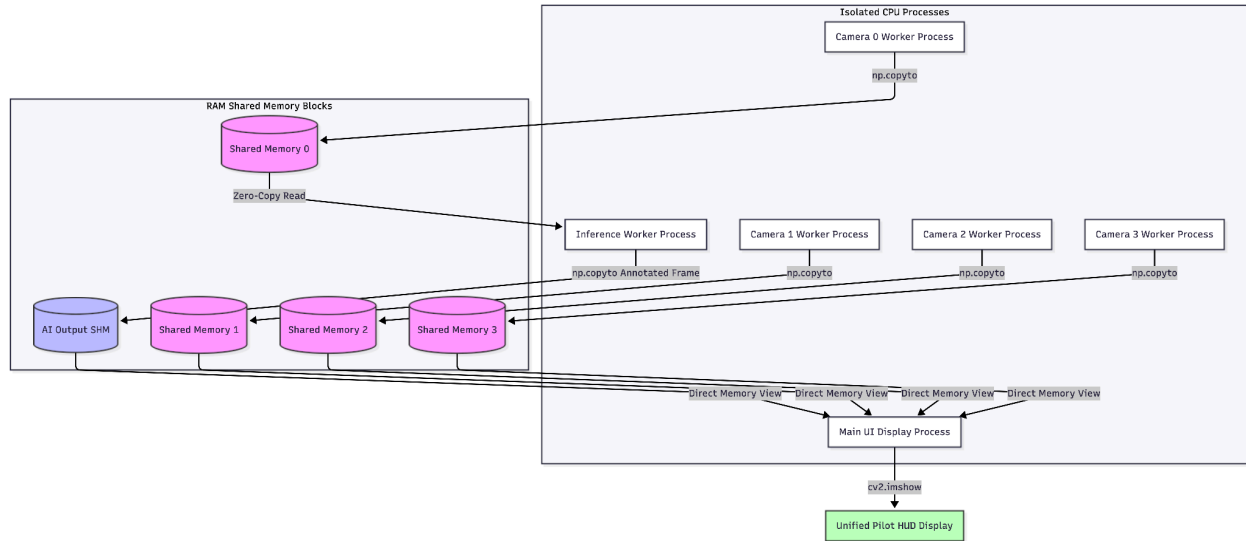


Figure 3.1a: Signals consist of integer frame ticks and float timestamps; data blocks match fixed $1920 * 1080 * 3 \sim$ dimensions.

4. Technical Performance Metrics (Telemetry Averages)

During the testing phase, a background telemetry logger samples system states every 10 seconds to generate a historical flight recorder. The compiled arithmetic means across all validation runs indicate excellent stability under heavy computational loads:

Metric Allocation	Calculated Average	System Performance Assessment
Pure AI Inference Time	18.52 ms	Excellent throughput consistency; CPU pipeline handling YOLO layers unhindered.
Total AI Stream Latency	56.09 ms	Stabilized under Multi-Processing; no queue or lock delays building up.
Auxiliary Feed Latency (Raw 1-3)	49.88 ms	Independent workers decoding video frames concurrently outside the main loop.
Global System Loop Speed	54.12 FPS	Smooth, high-frequency navigation HUD display provided to the pilot.

Figure 3a Performance plot charting System Latency vs. Pure Inference Time over 10 minutes to verify multi-process stability under load.

5. Connection to Mission Objectives

5.1.1 Reduction in Pilot Cognitive Load & Streamlining Pilot Experience: The display stays locked at a consistent rate, yielding a dependable real-time spatial awareness map for precision alignment tasks. The multi-processing shared memory model decouples the primary AI camera from the auxiliary camera feeds. If a complex visual scene drives up inference processing time, the auxiliary inspection loops stream unhindered at full speed, eliminating visual synchronization errors during tight vehicle maneuvers.

5.1.2 Mission Success Improvements: Traditional camera driver lockups require a full topside software reboot or vehicle surfacing, which squanders battery life and invalidates active runs. The autonomous watchdog detects and clears driver stalls within ~ 1.15 seconds, recovering lost feeds automatically to preserve mission continuity. Operational pilots validated a significant increase in tactile control precision due to the absolute consistency of the auxiliary tracking displays, eliminating variable latency spikes and providing reliable relative velocity mapping.

6.0 Computer Vision & Crab Image Detection

6.1 Abstract

Slugbotics developed a custom deep learning pipeline using the YOLO11 architecture (yolov11n.pt). Processing at a designated 640×640 resolution, the lightweight 2.6M parameter model puts a computational load of exactly 6.4 GFLOPs per frame on the central processing pipeline, maintaining a stable multi-camera stream on the CPU. A programmatic data augmentation pipeline was engineered using the Albumentations library to simulate underwater pool distortions. To eliminate Inter-Process Communication (IPC) bottlenecks and Python Global Interpreter Lock (GIL) constraints, the pipeline utilizes an isolated multi-process producer-consumer architecture communicating via POSIX shared memory allocations. Real-time telemetry integrity is preserved through an independent watchdog loop and metric logger, ensuring reliable, zero-allocation counting of invasive European Green crabs within a 0.5m to 1.2m operating envelope.

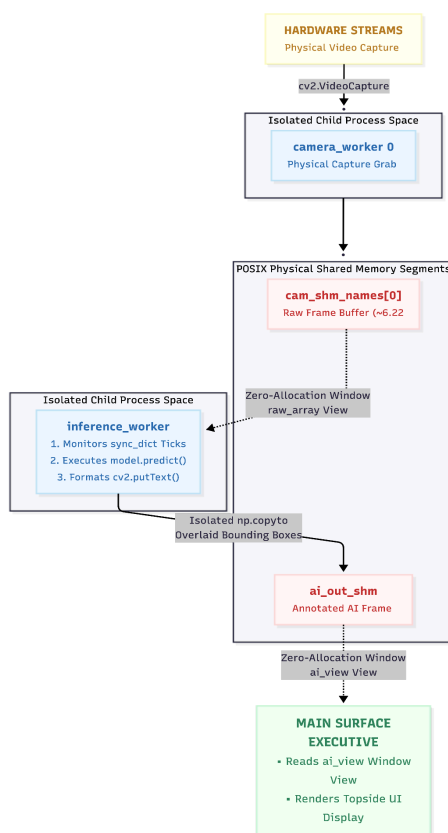
6.2 Multi-Process Producer-Consumer Architecture

Slugbotics allocated dedicated byte slots directly in physical RAM using Python's `shared_memory.SharedMemory` (Frame Size=1080×1920×3 bytes=6,220,800 bytes ~6.22 MB

per frame). The `inference_worker` wraps this raw segment as a local `np.ndarray` window view (`raw_array`). This allows the model to execute predictions directly from the memory buffer written by the camera capture process, achieving zero-allocation data access.

Synchronized Sequence Tracking & CPU Throttling

1. To prevent the inference process from spinning continuously on duplicate data, frame state synchronization is handled via an integer counter inside a shared multiprocessing. The inference process reads `sync_dict["frame_tick_0"]`. An evaluation is executed if current frame tick diverges from the worker's tracked state (`current_tick != last_tick`). **2** If a new frame has not yet arrived, the worker enters a micro-sleep state (`time.sleep(0.002)`). This rate-limits the waiting cycle, preventing unnecessary CPU core utilization and thermal throttling. **3.** Once `model.predict()` resolves, the resulting bounding box boundaries and classification target counts are overlaid directly onto the matrix via OpenCV. **4.** The final annotated array is passed back to the master topside process using an in-place `np.copyto` command into the pre-allocated `ai_out_shm` region. **5.** When an operator saves an inspection image (`'p'`), `cv2.imwrite` reads directly from the



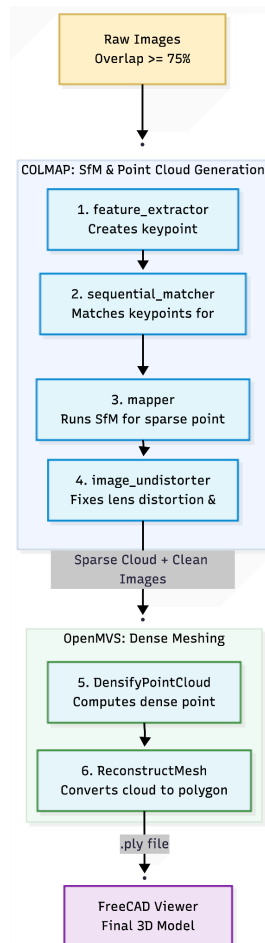
ai_viewreference, routing the image data straight to disk without needing high-overhead array extraction or duplication steps.

7.0 Photogrammetry

7.1 Abstract

Slugbotics uses a pipeline for 3D modelling the ROV's environment by using COLMAP, an open source Structure-from-Motion (SfM) and Multi-View Stereo (MVS) photogrammetry tool. COLMAP gathers 2D images and converts them into a sparse 3D point cloud. COLMAP is then paired with OpenMVS to complete the dense meshing, constructing polygons to create a 3D model in a .ply mesh file, which is then analyzed on FreeCAD. To accommodate CPU-only usage, Slugbotics uses COLMAP instead of other software, as COLMAP is natively made for Linux, which is the OS of our topside laptop.

7.2 Photogrammetry Pipeline



Due to the lack of a GPU, the reconstruction is split between COLMAP (SfM / point cloud) and OpenMVS (dense meshing).

6.3 Underwater Limitations

Photogrammetry has an apparent difficulty with rendering objects that have a shiny and reflective finish. It confuses the software's feature matching, often interpreting reflection as empty spaces. The main concern is that an underwater environment refracts light. To counteract this occurrence, Slugbotics mounted a white light source facing in the same direction as the main camera to produce flat lighting onto the coral.

6.4 Performance Benchmarking

The process time for photogrammetry averages between 8 and 15 minutes, depending on the object's complexity and image count. Creating a 3D model of a tree took 9 min 32 sec, with a total of 26 reference images taken. To create the model as fast as possible, we reduced the image quality to 1080p, skipped adding texture to the mesh, and set the image matching to sequential since the images will be organized in order.

- Average Image Count: 28.3 images
- Average Render Time 10 min 10 sec

Figure 2a Photogrammetry Pipeline

8.0 Control System: Manipulator Actuation Software

8.1 Software Design & System Integration

Slugbotics designed a custom Python-based software module to actuate the ROV's arm/claw. Rather than relying on standard PWM passthrough, we opted for a robust software-based control system integrated directly with the vehicle's BlueOS MAVLink router. The ClawController architecture utilizes the pymavlink library to establish a UDP connection capturing telemetry and command data. The primary advantage of this software design is its ability to seamlessly intercept MANUAL_CONTROL MAVLink messages directly from the topside controller.

8.2 Operator Performance Enhancements

The software implements state-tracking toggle logic rather than a momentary hold. The MANUAL_CONTROL message array is continuously polled, and a bitwise operation isolating the 4th bit ($1 \ll 3$) checks for the "Y" button input. By implementing a debounce and state-tracking flag (y_was_pressed), the pilot only needs to press "Y" once to lock the claw open, and press it again to lock it closed. This logic significantly reduces pilot cognitive load and hand fatigue during complex mission tasks requiring prolonged object manipulation.

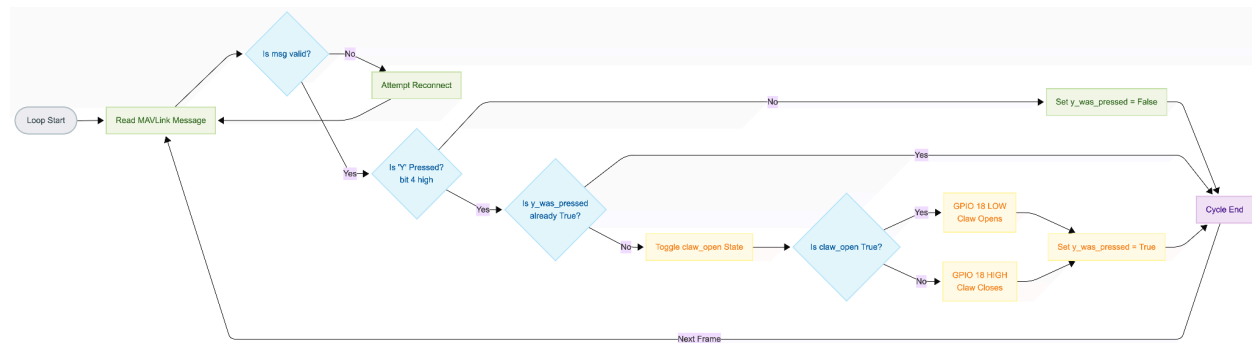


Figure 3.2a The progression from data ingestion (left) through debouncing logic (center), ending in physical GPIO manipulation.

8.3 Safety Fallbacks and Troubleshooting Equipment

Operational safety was the primary driver in our software development lifecycle. The script features robust troubleshooting strategies and error handling to protect the manipulator hardware and maintain mission uptime. **1. Fail-Safe Initialization:** Upon startup, the GPIO pin is explicitly forced HIGH (`GPIO.output(self.pin, 1)`). This ensures the manipulator relay defaults to a safe, unpowered state before the pilot takes control. **2. Connection Resilience:** The system relies on a `wait_heartbeat(timeout=10)` protocol to verify active communication with the vehicle. If the MANUAL_CONTROL stream drops, the software logs the null message and automatically triggers a `connect()` loop to re-establish communication without requiring a full system reboot, serving as an automated troubleshooting technique. **3. Graceful Degradation:** A dedicated `cleanup()` procedure intercepts manual keyboard interrupts or fatal loop errors. This routine forces the relay back to the safe HIGH state and clears all GPIO allocations, preventing erratic actuator behavior during an unexpected software crash.